

1. Python Language Fundamentals

Introduction

- Python is a high-level, interpreted, and easy-to-learn programming language created by Guido van Rossum in 1991.
- It is widely used in web development, automation, data science, artificial intelligence, and software testing.
- Python focuses on simple and readable syntax, which makes programming easier for beginners.

1.1 Introduction to Scripting

What is Scripting?

- Scripting means writing small programs to automate repetitive tasks like file handling and data processing.
- Scripts are executed line by line and are easy to write and modify.

Example of a simple script:

```
print("Welcome to Python")
print("This is a simple script")
```

Difference between Scripting and Programming

Scripting	Programming
Used for automation and small tasks	Used for building full applications
Interpreted and runs line by line	Usually compiled first
Faster to write	Takes more time to design
Example: Python, JavaScript	Example: C, C++, Java

1.2 History and Features of Python

Evolution of Python

- 1989: Python started by Guido van Rossum
- 1991: First public release
- 2000: Python 2.0 introduced
- 2008: Python 3.0 launched
- 2020: Python 2 stopped, Python 3 is now standard

Unique Features of Python

Feature	Explanation
Simple syntax	Easy to read and write
Interpreted	No need to compile
Platform independent	Works on Windows, Linux, Mac
Dynamically typed	No need to declare data type
Large library	Many ready-made functions

1.3 Python Pre-requisites

Required Software and Hardware

- Computer with minimum **4 GB RAM**
- Windows / Linux / macOS
- Python installed
- Any IDE like VS Code, PyCharm, IDLE

Basic Programming Knowledge

Basic understanding of:

- What is a program
- What is logic
- How to solve simple problems

1.4 Environment Setup for Python

Installing Python

Steps:

1. Download Python from python.org
2. Install it
3. Tick  **Add Python to PATH**
4. Check version using:

```
python --version
```

Setting up IDE

Common IDEs:

IDE	Use
VS Code	Best for general use
PyCharm	For big projects
IDLE	For beginners
Jupyter	For data science

1.5 Python Basic Datatypes

Main Data Types

Data Type	Meaning	Example
int	Whole numbers	10, -5
float	Decimal numbers	3.14, 2.5
str	Text	"Python"
bool	True / False	True

Example Program:

```
age = 20          # int
price = 99.5      # float
name = "Ram"      # string
is_student = True # boolean

print(age, price, name, is_student)
```

Type Conversion (Type Casting)

- Type conversion means changing one data type into another using functions like `int()`, `float()`, and `str()`.
- It is useful when handling user input and calculations.

Example:

```
x = "10"
y = int(x)    # converts string to int
z = float(y)  # converts int to float
print(y, z)
```

1.6 Python Variables, Keywords, Comments

Variables

- Variables are used to store data values in memory and can be changed during program execution.
- Python does not require declaring the data type of a variable.

Example :

```
name = "Amit"  
age = 21
```

Keywords

- Keywords are reserved words in Python that have special meaning, such as `if`, `else`, `for`, `while`, `True`, `False`, `break` and `continue`.
- These words cannot be used as variable names.

Comments

- Comments are used to explain the code and are ignored by the Python interpreter.
- In Python, comments start with the `#` symbol.

Example:

```
# This is a comment  
print("Hello")
```

1.7 Python Operators and Casting

Python Operators

- Python provides arithmetic, comparison, logical, and bitwise operators to perform different operations.
- These operators are used in calculations, conditions, and logical decisions.

Type	Example
Arithmetic	+, -, *, /
Comparison	==, >, <
Logical	and, or, not
Bitwise	&

Example:

```
a = 10
b = 3

print(a + b)    # Addition
print(a > b)    # Comparison
print(a > 5 and b < 5)  # Logical
```

Type Casting

- Type casting means converting one data type into another, such as converting int to float or string to int.
- It is done using built-in functions like `int()`, `float()`, and `str()`.

Example:

```
x = 5
y = float(x)
print(y)
```

1. Python I/O Expressions

Definition

Python I/O (Input/Output) expressions are used to **receive data from the user** and **display results on the screen**.

Diagram

User ⇌ Python Program ⇌ Screen

2. Input Statement – `input()`

Definition

`input()` is a built-in function used to **read data entered by the user from the keyboard**.

Diagram

Keyboard → `input()` → Variable

3. Reading Inputs of Different Data Types

Definition

The data received by `input()` is always a **string**, so it must be converted into other data types like **int** or **float** when needed.

Diagram

"25" → `int()` → 25
"3.5" → `float()` → 3.5

4. Formatting the Output – `print()`

Definition

`print()` is used to **display text, values, and results on the screen.**

Diagram

Data → `print()` → Screen Output

5. String Formatting Methods

Definition

String formatting is used to **combine text and variable values in a neat and readable way.**

Example:

```
name = "Tom"
print(f"My name is {name}")
```

Diagram

Text + Variable → Formatted String → Output

6. Python Control Statements

Definition

Control statements control the **order of execution** of a program.

Diagram

Program Execution → Controlled by Statements

7. Conditional Statements (if, elif, else)

Definition

Conditional statements are used to **make decisions based on conditions**.

Diagram

Condition → True Block / False Block

8. Nested Conditional Statements

Definition

A nested conditional statement is an **if statement inside another if statement**.

Diagram

```
if
├─ if
│   └─ Statement
```

9. Loop Control Statements (for, while)

Definition

Loops are used to **repeat a block of code multiple times**.

Diagram

Repeat → Repeat → Repeat → Stop

10. break, continue, pass

Definitions

Statemet	Meaning
break	Stops the loop completely
continue	Skips the current step
pass	Does nothing (placeholder)

Diagram

Loop

```
|— break → Exit loop
|— continue → Skip step
└— pass → No action
```

3.Python Control Statements

Introduction

Python control statements are fundamental building blocks used to control the flow of execution in a program. They help in making decisions, repeating tasks, and managing program logic efficiently. In this presentation, we will understand Python conditional statements such as `if`, `elif`, and `else`, along with loop statements like `for` and `while`. We will also explore loop control statements including `break`, `continue`, and `pass` with examples and flowcharts for better understanding.

Explanation

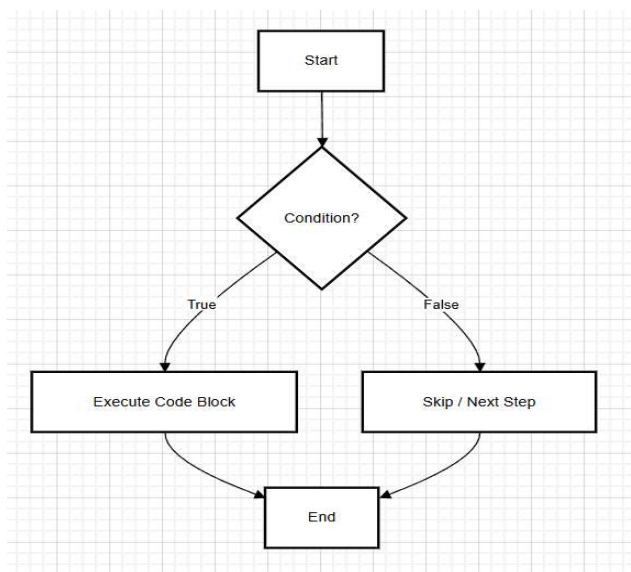
Python control statements control the execution flow of a program.

They decide which block runs, how many times it runs, and when it stops.

Types

- Conditional control statements
- Loop control statements

Flowchart



Python Conditional Control Statements

Explanation

Conditional statements are used to **make decisions** based on conditions.

If the condition is **True**, a block executes; otherwise, another block runs

◆ if, elif, else Statements

Explanation

- **if** checks the first condition
- **elif** checks another condition
- **else** runs if all conditions fail
Only **one block executes**.

Code Example

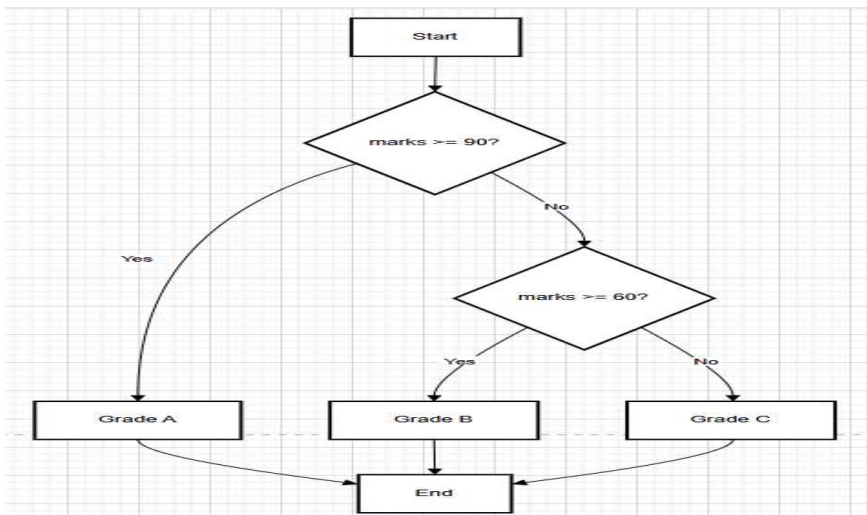
```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Output:

Grade B

FLOWCHART



◆ Nested Conditional Statements

Explanation

Nested conditionals mean if inside another if.

Used when decisions depend on multiple levels of conditions.

Code Example

```
age = 20
```

```
has_id = True
```

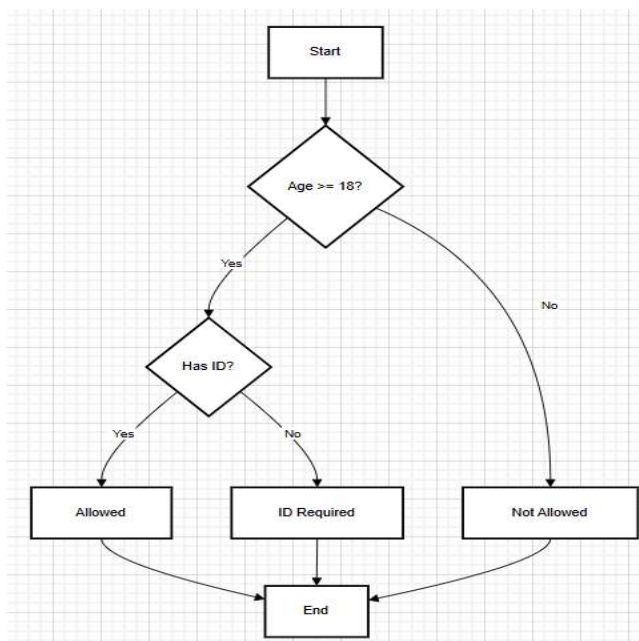
```
if age >= 18:
    if has_id:
        print("Allowed to enter")
```

```
    else:
        print("ID required")
else:
    print("Not allowed")
```

Output:

Allowed to enter

Flowchart



◆ Python Loop Control Statements

Explanation

Loops are used to repeat code multiple times.

Python supports **for** and **while** loops.

◆ for Loop

Explanation

A **for** loop iterates over a sequence like list, range, or string.

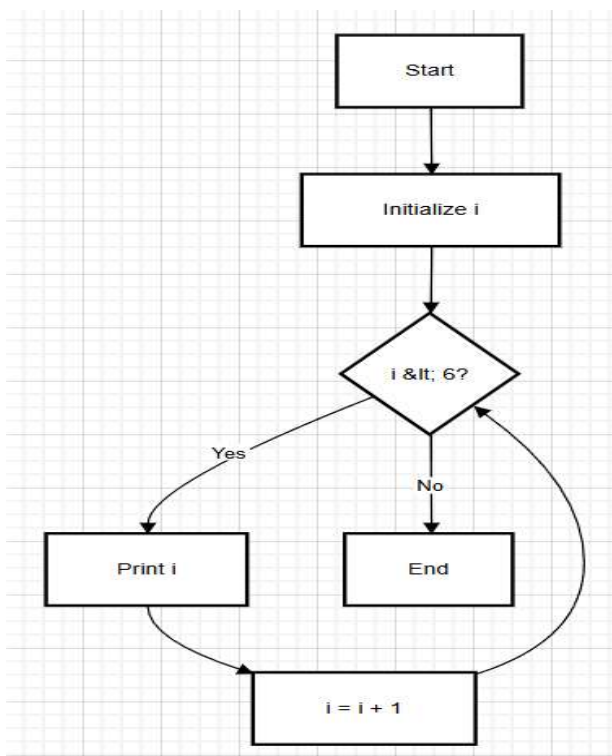
Code Example

```
for i in range(1, 6):  
    print(i)
```

Output

1
2
3
4
5

Flowchart



◆ while Loop

Explanation

A **while** loop runs as long as a condition is True.

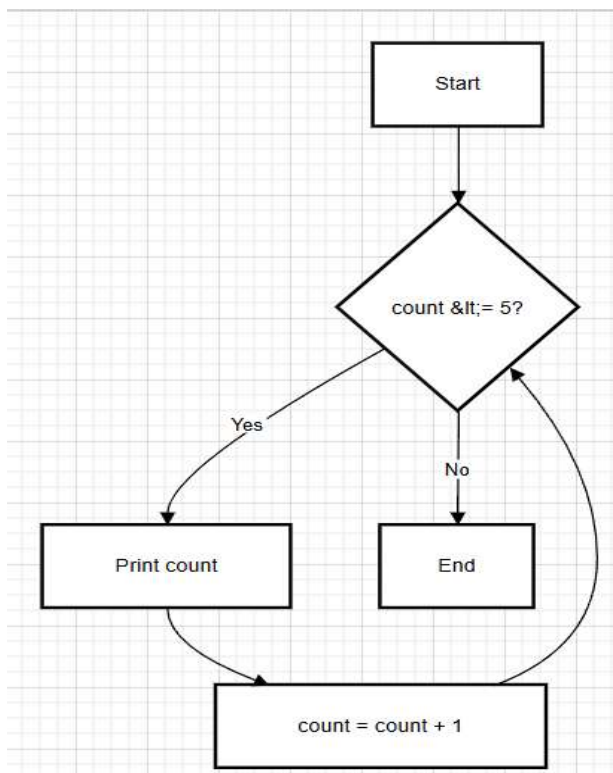
Code Example

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output

1
2
3
4
5

Flowchart



◆ break, continue, pass Statements

break

Explanation

`break` terminates the loop immediately.

Code:

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

Output

```
0  
1  
2  
continue
```

Explanation

`continue` skips the current iteration.

Code

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Output

0
1
3
4

pass

Explanation

`pass` does nothing and is used as a **placeholder**.

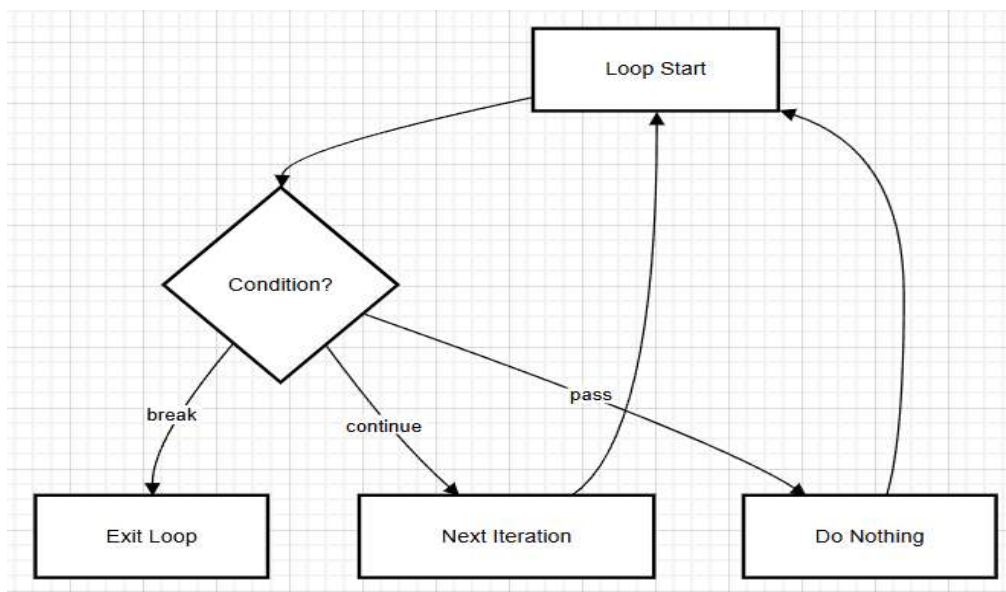
Code

```
for i in range(3):  
    Pass
```

Output:

(No output)

FLOWCHART



Conclusion:

In conclusion, Python control statements play a crucial role in writing logical and efficient programs. Conditional statements help in decision-making, while loops allow repeated execution of code based on conditions. Loop control statements like `break`, `continue`, and `pass` provide better control over loop execution. Understanding these concepts helps in developing structured, readable, and optimized Python programs.

Data Structures in Python

Introduction

In Python, **data structures** are used to store, organize, and manage data efficiently.

They allow us to store multiple values under a single variable name and perform operations like accessing, updating, adding, deleting, and iterating over data.

The main built-in data structures in Python are:

- List
- Tuple
- Dictionary
- Set

Each data structure has its own purpose and behavior.

1. List

What is a List?

A **list** is an ordered and mutable collection of elements.

This means:

- Elements are stored in a specific order
- Values can be changed after creation
- Duplicate values are allowed
- Different data types can be stored together

Example

```
numbers = [10, 20, 30, 40]
```

Accessing Elements in a List

List elements are accessed using **index values**, starting from 0.

```
numbers = [10, 20, 30, 40]
print(numbers[0])    # 10
print(numbers[-1])   # 40
```

Updating Elements in a List

Lists are mutable, so elements can be updated.

```
numbers = [10, 20, 30]
numbers[1] = 25
print(numbers)    # [10, 25, 30]
```

Slicing a List

Slicing is used to access a **portion of a list**.

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])    # [20, 30, 40]
```

Adding Elements to a List

Elements can be added in different ways.

```
numbers = [10, 20]
numbers.append(30)
numbers.insert(1, 15)
print(numbers)    # [10, 15, 20, 30]
```

Deleting Elements from a List

Elements can be removed by value or position.

```
numbers = [10, 20, 30]
numbers.remove(20)
numbers.pop()
print(numbers)    # [10]
```

2. Tuple

What is a Tuple?

A **tuple** is an ordered but immutable collection of elements.
Once created, its values **cannot be changed**.

Example

```
coordinates = (10, 20, 30)
```

Accessing Elements in a Tuple

```
coordinates = (10, 20, 30)
print(coordinates[1])    # 20
```

Slicing a Tuple

```
coordinates = (10, 20, 30, 40)
print(coordinates[1:3])    # (20, 30)
```

Updating Tuple Elements

Tuples do **not** support updating.

```
coordinates[0] = 50    # Error: tuple is immutable
```

When to Use Tuples

- When data should not change
 - When data integrity is important
 - For fixed configuration values
-

3. Dictionary

What is a Dictionary?

A **dictionary** stores data in **key–value pairs**, where each key maps to a value.

Example

```
student = {  
    "name": "Lokesh",  
    "age": 21,  
    "course": "Python"  
}
```

Accessing Dictionary Values

Values are accessed using keys.

```
print(student["name"])    # Lokesh
```

Updating Dictionary Values

```
student["age"] = 22  
print(student["age"])    # 22
```

Adding New Key–Value Pairs

```
student["grade"] = "A"
```

Deleting Elements from Dictionary

```
del student["course"]
```

Iterating Over a Dictionary

```
for key, value in student.items():  
    print(key, value)
```

When to Use Dictionaries

- When data needs a label
 - When fast lookup is required
 - When storing structured information
-

4. Set

What is a Set?

A **set** is an unordered collection of unique elements.
Duplicate values are automatically removed.

Example

```
marks = {80, 85, 90, 85}  
print(marks)    # {80, 85, 90}
```

Adding Elements to a Set

```
marks.add(95)
```

Removing Elements from a Set

```
marks.remove(80)
```

Accessing Set Elements

Sets do not support indexing.
Elements are accessed using loops.

```
for mark in marks:  
    print(mark)
```

When to Use Sets

- When duplicate values must be avoided
 - When performing membership checks
 - For unique collections
-

5. Working with For Loops on Data Structures

Purpose of For Loop

A **for loop** is used to iterate over elements of a data structure one by one.

For Loop with List

```
numbers = [1, 2, 3]  
for num in numbers:  
    print(num)
```

For Loop with Tuple

```
colors = ("red", "blue", "green")
for color in colors:
    print(color)
```

For Loop with Dictionary

```
student = {"name": "Lokesh", "age": 21}
for key in student:
    print(key, student[key])
```

For Loop with Set

```
values = {10, 20, 30}
for val in values:
    print(val)
```

6. Usage of **in** Keyword

What is the **in** Keyword?

The **in** keyword is used to check whether an element exists in a data structure.

in with List

```
numbers = [10, 20, 30]
print(20 in numbers)    # True
print(50 in numbers)    # False
```

in with Tuple

```
colors = ("red", "blue")
print("red" in colors)    # True
```

in with Dictionary

Checks only **keys**, not values.

```
student = {"name": "Lokesh", "age": 21}
print("name" in student)    # True
print(21 in student)        # False
```

in with Set

```
marks = {80, 90, 100}
print(90 in marks)    # True
```

Conclusion

Python data structures such as **List, Tuple, Dictionary, and Set** provide powerful ways to store and manage data.

Understanding how to **access, update, slice, add, delete, iterate**, and **check membership** forms a strong foundation for Python programming and real-world applications.

5. Python Strings

Definition of Python Strings

Strings are immutable sequences of characters enclosed in single (`' '`), double (`" "`), or triple (`' ' ' ' ' '`) quotes. They're Python's core text data type, supporting indexing, slicing, and built-in methods.

5.1 Slicing and Formatting Strings

Slicing Strings

Slicing allows you to extract a specific portion (substring) of a string using the syntax `string[start:stop:step]`.

- **Start:** The beginning index (inclusive).
- **Stop:** The ending index (exclusive).
- **Step:** The interval between characters (optional).

Python

```
text = "Python"
```

```
print(text[0:2]) # Output: 'Py' (Indices 0 and 1)
```

```
print(text[2:]) # Output: 'thon' (From index 2 to the end)
```

```
print(text[::-1]) # Output: 'nohtyP' (Reverses the string)
```

Formatting Strings

Modern Python provides two primary ways to inject variables into strings:

- **f-strings (Recommended):** Prefixed with `f`, these allow you to embed expressions directly inside curly braces `{}`. They are faster and more readable.
 - Example: `f"Hello, {name}!"`
 - **format() method:** Uses placeholders `{}` and passes values as arguments.
 - Example: `"User: {}".format(username)`
-

5.2 Built-in Methods

Python includes a variety of built-in methods to transform and analyze strings. Because strings are **immutable**, these methods always return a *new* string rather than modifying the original.

Common Methods

Method	Description	Example
lower()	Converts all characters to lowercase.	"Hi".lower() → "hi"
upper()	Converts all characters to uppercase.	"hi".upper() → "HI"
split(sep)	Breaks a string into a list based on a separator.	"a,b,c".split(",") → ['a', 'b', 'c']
join(list)	Concatenates a list into a single string.	"-".join(['a', 'b']) → "a-b"
strip()	Removes whitespace from both ends.	" hi ".strip() → "hi"
replace()	Replaces a substring with another.	"apple".replace("a", "o") → "opple"

Conclusion

Python strings provide powerful slicing, f-string formatting, and methods like `split()`, `join()`, `strip()`, and `replace()` for effective text processing in software testing and development projects.

6. Python Functions

Python functions help us **organize code**, **avoid repetition**, and **improve readability**. The given data mainly focuses on **built-in functions**, **range()**, and **user-defined functions**.

1. Built-in Functions and **range()** Function

◆ Built-in Functions

Built-in functions are **already available in Python**, so we can use them directly without writing extra code.

Common built-in functions:

- **len()** → Finds length of a string, list, tuple, etc.
- **type()** → Finds the data type of a variable
- **str()** → Converts data into string
- **int()** → Converts data into integer

◆ **range()** Function

The **range()** function is used to **generate a sequence of numbers**.

Syntax : `range(start, stop, step)`

Example:

```
for i in range(1, 6):  
    print(i)
```

Commonly used in:

- Loops
- Counters
- Iterations

2. User-Defined Functions

User-defined functions are **created by programmers** to perform a specific task.

◆ Creating Functions

Functions are created using the `def` keyword.

Syntax:

```
def function_name(parameters):  
    statements  
    return value
```

Example:

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

◆ Parameters and Return Values

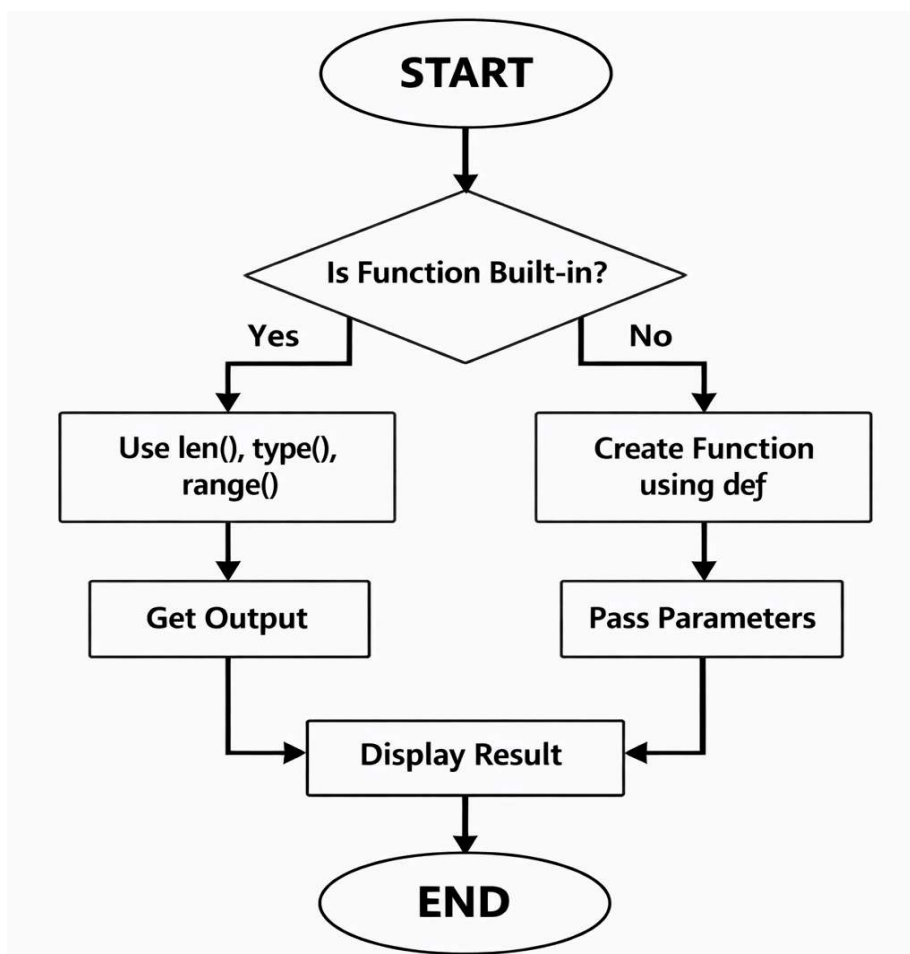
- **Parameters:** Values passed to a function.
- **Return value:** Output given back by the function using `return`.

Benefits:

- Code reusability
- Easy debugging
- Better structure

Flowchart:

Python Function Working Process



GROUP 1 Members topics and sub topics

NAME	TOPICS	SUB TOPICS
57152-Abhishek Suman	Python language Fundamentals	● Introduction to Scripting ● History and Features of Python ● Python Pre-requisites ● Installing Python ● Type conversion ● Python Variables, Keywords, Comments ● Type casting in Python ● Data Types Overview
55923-Pavithra Gaddam	Python I/O Expressions	● Inputting Statements ● Using the input() function ● Reading inputs of different datatypes ● Formatting the Output ● Using the print() function ● String formatting methods
55926-Konijeti Yamini	Python Control Statements	● Python Conditional Control Statements ● Understanding if, elif, and else statements ● Nested conditional statements ● Python Loop Control Statements ● Working with for and while loops ● Understanding break, continue, and pass statements
55903-Lokesh Prabhu	Data Structure in Python	● List, Tuple, Dictionary, Set (Accessing, Updating, Slicing) ● Understanding lists, tuples, dictionaries, and sets ● Accessing and updating elements ● Slicing data structures ● Adding and Deleting Elements ● Methods to add and remove elements ● Working with For Loops

		• Iterating over data structures • Usage of 'in' Keyword • Checking membership using 'in'
55915-Ritikesh Kumar	Python String	• Slicing and Formatting Strings • Accessing substrings • Formatting strings using f-strings and format() • Built-in Methods • Common string methods like lower(), upper(), split(), join()
55913-Himaja Sudha	Python Functions	• Built-In Functions and Range() Functions • Understanding built-in functions like len(), type(), str(), int() • Generating sequences using range() • User Defined Functions • Creating functions • Understanding parameters and return values